2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge) | 979-8-3315-0211-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/Forge66646.2025.00009

RepoHyper: Search-Expand-Refine on Semantic Graphs for Repository-Level Code Completion

Huy N. Phan
FPT Software AI Center
Viet Nam
huypn16@fpt.com

Hoang N. Phan
Department of Computer Science
Nanyang Technological University
Singapore
C210055@ntu.edu.sg

Tien N. Nguyen
Computer Science Department
The University of Texas at Dallas
USA
tien.n.nguyen@utdallas.edu

Nghi D. Q. Bui
FPT Software AI Center
Viet Nam
bdqnghi@gmail.com

***Abstract*—Code Large Language Models (CodeLLMs) have demonstrated impressive proficiency in code completion tasks. However, they often fall short of fully understanding the extensive context of a project repository, such as the intricacies of relevant files and class hierarchies, which can result in less precise completions. To overcome these limitations, we present REPOHYPER, a multifaceted framework designed to address the complex challenges associated with the completion of code at the repository level. Central to REPOHYPER is the *Repo-level Semantic Graph* (RSG), a novel semantic graph structure that encapsulates the vast context of code repositories. Furthermore, REPOHYPER leverages *Expand and Refine* retrieval method, including a graph expansion and a link prediction algorithm applied to the RSG, enabling effective retrieval and prioritization of relevant code snippets. Our evaluations show that REPOHYPER markedly outperforms existing techniques in repository-level code completion, showcasing enhanced accuracy across various datasets when compared to several strong baselines.**

***Index Terms*—Repo-level Code Completion, Repo-level Semantic Graph, Expand-Refine Retrieval**

I. INTRODUCTION

The advent of AI-assisted code completion tools marks a significant milestone in software development. These tools, while adept at interpreting the immediate context of the code being written, often do not fully exploit the broader context available within the entire code repository. This oversight results in suggestions that might not be optimally aligned with the project's architecture or intended functionality, as these tools tend to overlook the rich information embedded in related files, class hierarchies, dependencies, and more.

To overcome these shortcomings, a direct but complex solution involves enhancing the context length of language models by applying efficient attention techniques [1]–[4]. Nonetheless, increasing the context length significantly raises costs and is not feasible indefinitely, especially with respect to the voluminous number of files in a given repository. Hence, there is a crucial need for more refined strategies that accurately identify relevant contexts rather than indiscriminately analyzing every file within a repository. In response to this challenge, the concept of repository-level code completion has gained traction. It aims to incorporate the full context of a project, including inter-file relationships, imported modules, and the overarching project structure [5]–[9]. These methodologies generally employ a similarity-based approach to retrieve contexts for completing a given code snippet, drawing either from raw source code or a pre-constructed database with essential metadata. However, this strategy exhibits significant limitations. It often fails to consider that diverse contexts within the current repository, not necessarily involving similar code, can provide valuable insights for code completion. This includes the intricate network of dependencies, shared utility functions, inter-module method calls, class hierarchies, inter-class dependencies, and encapsulation patterns—all of which are fundamental to program semantics.

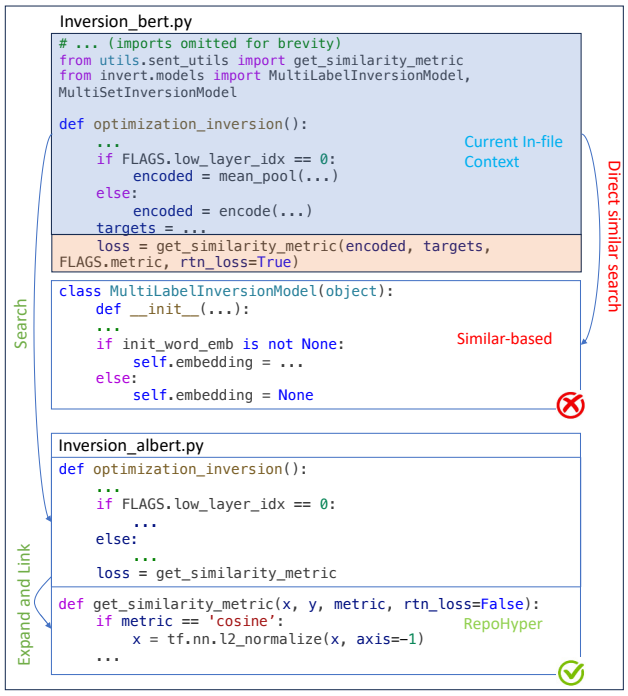

Fig. 1. Illustration of graph-based semantic search versus similarity-based search. The orange block indicates the ground-truth line that needs to complete to call the function *get_similarity_metric*. Similarity-based methods mistakenly focus on *MultiLabelInversionModel* class due to its similarity in form with current in-file context, leading to incorrect completions. Conversely, REPOHYPER successfully identifies the correct context via first identify the most similar code snippet in the codebase then expand and link.

979-8-3315-0211-9/25/$31.00 ©2025 IEEE
DOI 10.1109/Forge66646.2025.00009

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

To address these shortcomings, we introduce REPOHYPER, a novel repository-level code completion approach that considers global, repository-level contexts including not only similarity code but also program-semantic related contextual information within the current repository. Specifically, REPOHYPER incorporates three following components: (1) **Repo-level Semantic Graph (RSG)**, which is a graph-structure designed to encapsulate the core elements of a repository's global context and their dependencies pertaining to code completion, serve as a reliable knowledge source to retrieve accurate contexts instead of raw codebase; (2) **Expand and Refine** retrieval method which consists of two steps: (2.1) **Search-then-Expand Strategies**,which broaden the exploration of contexts by identifying semantically similar ones and then expanding the search to include contexts semantically linked, utilizing the RSG for intelligent navigation and (2.2) **Link Predictor** which is a mechanism that refines the broad set of contexts obtained from the *Search-then-Expand Strategies* and prioritizes them in a smaller, highly relevant subset for code completion. This is accomplished by formulating the re-ranking problem as a link prediction within the RSG, thereby reducing distractions for the LLM.

We conducted several experiments to comprehensively evaluate REPOHYPER on both **context retrieval (CR)** and **end-to-end code completion tasks (EECC)** using the RepoBench benchmark [10], demonstrating significant improvements over existing state-of-the-art methods. In CR, REPOHYPER outperforms similarity-based approaches by an average of 49% in retrieval accuracy, utilizing the same encoder [11], [12]. For EECC, our method surpasses RepoCoder and other RepoBench baselines, achieving an improvement of +4.1 in Exact Match (EM) and +5.6 in CodeBLEU scores.

To summarize, in this paper, we make the following key contributions:

1) We introduce **REPOHYPER**, a novel framework featuring three novel modules designed to address the multifaceted challenges of repository-level end-to-end code completion.
2) We develop the **RSG**, a novel graph representation that captures the global context of a repository, expanding to include non-similar but yet relevant contexts for repo-level code completion. This innovation significantly improves the accuracy and relevance of context retrieval, surpassing conventional methods.
3) We implement an **Expand and Refine** retrieval method via **Search-then-Expand Strategies** and **Link Prediction algorithm** within the RSG, optimizing the retrieval of the most relevant and program-semantic related contexts.
4) We perform extensive evaluation of REPOHYPER in both repository-level code retrieval and code completion tasks demonstrates a significant improvement over the state-of-the-art approaches. Through a series of analytical and ablation studies, we confirm the vital role of each component of REPOHYPER.

II. RELATED WORK

A. Code LLMs for Code Generation & Understanding

Recent research has introduced a plethora of Large Language Models (LLMs) tailored for code-related tasks [11], [13], [13]–[26], aiming to enhance code understanding and generation. These models are categorized into closed-source and open-source variants. Initially, closed-source models like Codex [15], Code-Davinci [15], and PaLM-Coder [14] demonstrated exceptional performance on well-known code completion benchmarks, including HumanEval [15], MBPP [16], and APPS [17]. Subsequently, the emergence of open-source models such as the CodeGen series [18], [19], CodeT5 [27], CodeT5+ [11], CodeGeeX [20], StarCoder [24], Wizard Coder [28], CodeLlama [29], and DeepSeek-Coder [23] began to rival the closed-source models in terms of benchmark performance. Despite their purported efficacy across a broad spectrum of code intelligence tasks, code generation and completion emerge as their most notable and widely utilized applications.

Notably, decoder-only transformer models trained on extensive code corpora have marked a leap forward in this field. Recent examples, such as Codex [15], DeepSeek-Coder [23], and CodeLlama [29], leverage billions of parameters to achieve remarkable code generation performance, benefiting from vast, high-quality code databases to support programmers.

B. Repository-level Code Completion

Repository-level code completion has advanced beyond simple similarity measures. For instance, RLPG [30] and CoCoMIC [31] enhance code completion by integrating richer structural and contextual signals, yet they rely on static or hand-crafted context extraction. Similarly, RepoFusion [7] and related methods assume the ready availability of external context sources. Methodologies like RepoFusion [7], RepoPrompts [9], and MGD [8] focus on integrating diverse context sources—often assuming that external or pre-aggregated context is readily available. RepoFusion, for example, demonstrates the potential of fusing multiple context cues but depends on external inputs, which can limit its adaptability in real-world scenarios where such information might be incomplete or inconsistent. In contrast, REPOHYPER is designed to overcome these limitations by adopting a fully automated, end-to-end framework that jointly models both granular code elements and high-level repository structure.

Benchmarks such as RepoBench [10] and CrossCodeEval [32] underline the need for comprehensive evaluation frameworks in repository-level scenarios. Our detailed comparisons with prior works—specifically RLPG, CoCoMIC, and RepoFusion—highlight that while these methods have made significant contributions, RepoHyper's novel integration of automated context extraction with a multi-agent framework provides superior adaptability and accuracy. Furthermore, by addressing both within-file and cross-file challenges simultaneously, REPOHYPER represents a substantial step forward in achieving truly scalable and context-aware code completion.

Additionally, multi-agent systems like CodeAgent [33] and HyperAgent [34] introduce mechanisms for integrating diverse

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

forms of contextual information (e.g., documentation, runtime environments) into the code completion pipeline. Unlike these approaches, which focus on external or runtime contexts, RepoHyper exclusively targets the inherent structural and semantic complexities of the repository itself, thereby offering a complementary solution that emphasizes automated and holistic context integration.

Moreover, these methods often require tuning parameters such as chunking size and overlap size, which are not inherently suited to the source code domain. They lack customization for handling unique characteristics of source code, such as symbol segmentation and the relationships between symbols.

To address these limitations, our approach constructs a global repository-level graph, where each major symbol in the codebase is represented as a unique node. This graph-based representation enables capturing intricate relationships across the repository, facilitating more comprehensive and accurate code completion.

C. Learning on Graph Representations of Code

Graph Neural Networks (GNNs) have gained significant traction across various domains due to their ability to process and analyze graph-structured data. In particular, they have demonstrated impressive performance in fields such as computer vision [35] and natural language processing [36], [37]. Their inherent capability to model relationships and structures has also made them a promising tool in source code analysis, where code naturally exhibits graph-like properties. Studies have leveraged GNNs for a range of code-related tasks, including variable naming and misuse detection [38], as well as vulnerability identification [39]. Among these efforts, GraphGen4Code [40] represents a notable approach, constructing knowledge graphs for code at the file level by employing generic techniques to capture program semantics as graphs. However, file-level contexts, as built by GraphGen4Code, often fall short when addressing repository-level tasks. These limitations underscore the need for a more comprehensive graph representation that captures the global, repository-level context essential for tasks like code completion.

III. RepoHyper Methodology

Figure 2 illustrates the overall architecture of our approach, RepoHyper. Given an existing, incomplete code snippet Q, we first encode it into a semantic vector using an encoder function. Our objective is to retrieve relevant semantic contexts T from the repository R. Consequently, these retrieved contexts are subsequently utilized by a Large Language Model (LLM) to generate the final code prediction:

$$C = \text{LLM}(Q, T).$$

We present a *Repo-level Semantic Graph* (RSG) for global context representation (Section III-A) and a *Expand and Refine* retrieval algorithm to re-rank and select relevant snippets from RSG. This includes two key steps: **Search-then-Expand** which tries to find the most similar and program-semantic related contexts, and **Re-ranking as Link Prediction** which aims to refine the contexts set found by the prior step. (Section III-B and Section III-D).

A. Repo-level Semantic Graph (RSG): Representation for Global Contexts

1) Relation Representation

Our main goal is to design a graph representation that captures the fundamental units of the repository-level context in a project and intricate relations among them. In our design, we choose *function/method* and *class* as the fundamental units because they are basic for computation in a program. Each function or method defined in the project is represented as a unique node. The node's content includes the function/method's name, its parameters, and implementation. This ensures not only the fine-grained information access for code in LLMs since most of functions/methods usually has small context length but also the exact segmentation between contexts eliminating the need of manual tuning, chunking, and overlapping size. Furthermore, it also guarantees a distinct edge for every instance of a function call or for delineating the ownership connections between classes and methods, etc which are all significantly necessary for repository-level completion [41].

2) Context Problem

Nodes also represent all the classes declared within the project contain the entire class implementation. However, since class implementations often exceed the maximum context length of the base model we use for later retrieval, we limit the node's content to the class name and its initialization method, if present. Importantly, upon identifying all functions and classes in the project, we remove them from the files. Subsequently, each file retains only the import statements and other non-function or non-class statements. We encapsulate these remaining statements as a passage for each file and represent them with a *Script* node.

3) Design and Implementations

In the realm of context modeling for code LLMs, a pivotal consideration is the granularity of the context and relationship between programming language specific objects like functions and classes.

To realize this objective, we represent each major object of the programming language as an independent node. This approach not only ensure the fine-grained information access for code LLMs since most of functions, methods in our experiment usually has small context length but also ensure the exact segmentation between contexts eliminating the need of manual tuning chunking and overlapping size. Furthermore, it also guarantees a distinct edge for every instance of a function call or for delineating the ownership connections among classes and methods, etc which are all significantly necessary for repository-level completion which is found by [41]. Nodes within our graph may represent functions, methods, classes, or scripts for Python.

We denotes Repo-level Semantic Graph (RSG) as $\mathcal{G} = (V, E)$, where V denotes a set of nodes and E denotes a set of relations, aims to capture the fundamental units of a project's global context and the intricate relationships among them. We

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

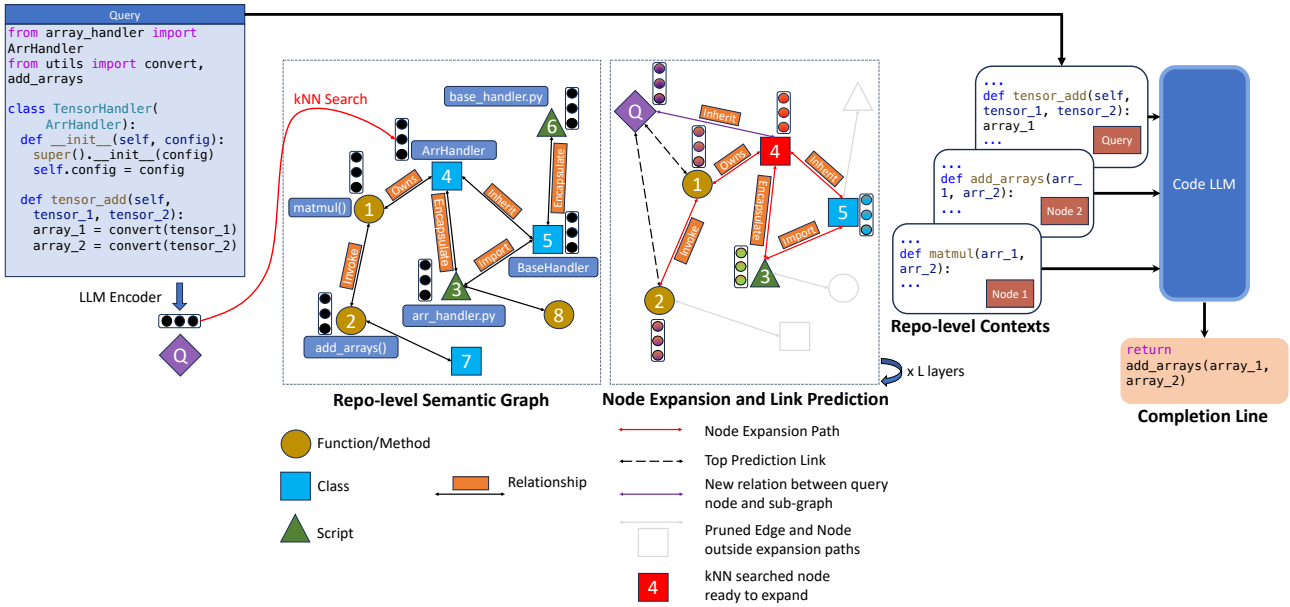

Fig. 2. Overall Architecture of REPOHYPER

consider (1) *function/method* and (2) *class* to be fundamental units due to their crucial role in program structure. Each node contains the name, parameters, and body of the corresponding function/method. This allows for precise context access and clear separation of function calls and class-method relationships, which is crucial for repository-level code completion [41]. This also ensures precise context segmentation, eliminating the need for manual chunking and size tuning. After extracting functions and classes, the remaining file content, such as import statements and non-functional code, is encapsulated in (3) a *Script* node.

The nodes in an RSG are interconnected based on their types and relations, which are categorized as follows: (1) *Import Relations*: These relations (Imports and Imported By) exist between script nodes and the imported modules identified from the script's import statements. This excludes external modules not within the project's scope; (2) *Invoke Relations*: These relations (Caller and Callee) exist between functions (or methods) when one node invokes another; (3) *Ownership Relations*: These relations (Owns and Owned By) exist between methods and the classes that contain them; (4) *Encapsulate Relations*: These relations (Enclose and Enclosed By) exist between script nodes and other nodes that have code snippets contained within the file represented by the script node; and (5) *Class Hierarchy Relations*: These relations (Inherits and Inherited By) exist between classes.

While our implementation is for Python, RSG can be adapted for using in other high-level programming languages, e.g., Java or C++. This adaptation involves utilizing different program elements as context units, and their relations in an RSG. A specific RSG for a new programming language needs to be constructed accordingly in the same framework.

B. Search-then-Expand Strategies

Our methodology employs a search-then-expand approach to identify the most suitable file for the decoding process within a repository context. We aim to broaden the search space beyond similarity-based candidates to encompass more relevant files, as suggested by [5], [10], [41]. These studies indicate that files with similar imports, names, or code snippets are typically the correct contexts for retrieval, and that semantic search using k-Nearest Neighbor (kNN) with encoders like UniXCoder [12] or CodeT5+ [11] is effective. Additionally, they highlight the importance of structured context sources such as *Sibling* files or *Import of Parent Class* in providing relevant contexts.

Based on these insights, REPOHYPER initially performs a kNN search with a small K to identify a set of anchor nodes in the RSG. These nodes are then expanded using strategy $\mathcal{F}$:

$$A_{\text{exp}} = \mathcal{F}(A), \quad A = \{V_i | i \in \text{kNN}(\mathcal{G}, Z_Q)\}$$

$\text{kNN}(\mathcal{G}, Z_Q)$ represents the kNN search for the K nodes most similar to the query vector Z_Q in the graph $\mathcal{G}$. We experiment different strategies to find the most optimal nodes in the graph for decoding process. Our two proposed strategies are:

(1) *Exhausted Search*: Beginning from a node $V_j \in A$, we utilize a straightforward Breadth First Search algorithm (BFS) with a maximum depth of D. Theoretically, D should reach 4 to encompass the complete relationship between two contexts for repo-level code completion. However, in practice, setting $D \geq 3$ may result in the BFS covering nearly 50% of the graph. Hence, we introduce another parameter alongside D to constrain the number of BFS expanded nodes: the maximum number of nodes per BFS denoted as M.

(2) *Pattern Search*: We found that exhausted search on all the directions and paths can include too many irrelevant contexts

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

to the query. Thus, we conduct kNN search for all the queries in the training set, then expand using exhausted strategy from kNN searched node V_j to target node V_{target}, then collect the most frequent type paths into path set $\mathcal{P}$, then add them as filters for later BFS. This is called as pattern search since it will eliminates non-frequent paths during exhausted exploration saving the walking nodes.

$$A_{\text{exp}} = \{V_i | V_i \in \mathcal{F}_{\text{exh}}(A), \text{PATH}(V_j, V_i) \in \mathcal{P}\}$$

where PATH denotes the type of BFS exploration path from kNN searched node $V_j \in A$ to any walking node V_i. For example, one might prioritize the exploration from a class node to its script node then to imported function to draw a possible invoke relationship for code completion (class-script-function path type), but one is unlikely to explore chain of method calls (method-method-method path type).

C. Pattern Search

In Repository Semantic Graph (RSG), there are three types of nodes: function (method), class, and script. The edges between these nodes represent five different relationships: import (`1`), invoke (`2`), ownership (`3`), encapsulate (`4`), and hierarchy (`5`). A path type is defined by the sequence of edge types along that path in the RSG. For example, let's consider a node V_j identified by the k-nearest neighbor (kNN) search, and a target node V_{target} representing the "gold snippet" (e.g., a method called by V_j). There can be multiple paths from V_j to V_{target}. One such path could be: V_j (method) $\rightarrow$ (owned by) V_1 (class) $\rightarrow$ (encapsulated in) V_2 (script) $\rightarrow$ (import) V_{target} This path has a type of (ownership, encapsulate, import), represented by the edge types (3, 4, 1). However, since there are multiple paths from V_j to V_{target}, including all the nodes produced by these paths might introduce irrelevant contexts into the subgraph. For example, V_j can reach V_{target} through a more useful path like: V_j (method) $\rightarrow$ (owned by) V_1 $\rightarrow$ (owns) V_3 (method) $\rightarrow$ (call) V_{target} This path is more useful because V_3 (method) can provide hints on how to call V_{target}, whereas the previous path included V_2 (script), which is a longer context and less useful for understanding how to use V_{target}. In the training set of RepoBench-R, for each sample, we have a gold snippet and an in-file context. We first use kNN search to find the k most similar context nodes to the in-file context. Then, we perform an exhaustive search from these identified nodes to collect all possible paths leading to the gold snippet. For example, in sample 1, we might collect paths of types (3, 4, 1) and (1, 2, 3), while in sample 2, we might find paths of types (3, 4, 1) and (2, 4). The most frequent path type in the training set is (3, 4, 1), which appears twice in this example.

D. Re-ranking as Link Prediction

The *Search-then-Expand strategy* increases the number of potential contexts by retrieving and expanding a subgraph around the query. However, this introduces the challenge of potential noise from irrelevant nodes, which could impair the model's performance. To address this, we refine the context selection process by focusing on the **top-**N_2 most relevant contexts, where $N_2 < N_1$. This filtering ensures that the decoder processes only the most meaningful nodes, reducing unnecessary computation and improving prediction quality.

Initially, contexts were ranked based on their *embedding distances* to the query. However, this heuristic approach underperformed in evaluation due to its inability to fully capture the complex relationships and dependencies within the graph. To address this, we reformulated the relevance ranking as a **link prediction problem** on a **Repo-level Semantic Graph**.

In this approach, the query is embedded as a node in the RSG. A **message passing network (MPN)**, equipped with a link prediction head, is then employed to score the connections between the query node and other nodes in the graph. These scores represent the relevance of each context node to the query, allowing us to rank them effectively.

The relevance scoring process is powered by the MPN, which refines node embeddings through iterative message passing. The *final embeddings* produced by the MPN are used to compute linking scores that quantify the strength of relationships between the query and other nodes:

$$Z_i^{(L)} = f(Z_i^{(0)}, \mathcal{G} \oplus \mathcal{Q})$$

Here:

- $Z_i^{(0)}$: The initial embedding of the ith node, as generated by the encoder.
- $Z_i^{(L)}$: The final embedding of the node after L layers of message passing through the network f.
- $\mathcal{G} \oplus \mathcal{Q}$: Represents the combined graph, where the query node ($\mathcal{Q}$) and its edges are integrated into the extracted subgraph ($\mathcal{G}$).

It is essential to incorporate the **query node**, representing the in-file context, into the subgraph to ensure the extracted structure remains contextually complete. The query node provides localized code context, and its integration is pivotal for enabling meaningful message passing between itself and the nodes in the extracted subgraph. To facilitate this, edges connecting the query node to the subgraph must be established. Fortunately, in many cases, the query node already has inherent connections to some nodes within the subgraph. For example, in the task of *next-line prediction*, the query node (in-file context) might invoke certain functions or reference specific variables that are already represented as nodes in the subgraph. By integrating the query node and retaining these pre-existing connections, we enable the MPN to utilize both the localized and global context information effectively.

This integration is not merely structural but functional, as it ensures the query node contributes to the propagation of relevant information across the graph. This step allows the MPN to leverage both the syntactic and semantic relationships encoded within the graph, leading to better-informed predictions. Consequently, the process enhances the model's ability to capture the query-specific context, resulting in more accurate and contextually relevant *code completion suggestions*. By jointly considering the repository-derived subgraph and the

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

in-file context, the system can grasp intricate dependencies, leading to tailored and precise code recommendations.

The query node $\mathcal{Q}$ is initialized with raw source code data and an embedding Z_Q. It is added to the graph by augmenting the node set V and introducing edges that represent relationships relevant to the query. For example, if the query code snippet already invokes certain functions represented in the graph, edges are added to capture these relations, while avoiding duplicates.

By modeling the relevance among the program elements as a graph-based link prediction problem, this method effectively captures the structural and relational nuances of the query's context. The refined approach ensures that the top-ranked contexts are both relevant and comprehensive, enabling the decoder to produce code completion suggestions that are accurate, context-aware, and highly actionable.

$$s_i = W^T \text{concat}(Z_i^{(L)}, Z_Q^{(L)}) \quad \forall i \in \{i | V_i \in A_{\text{exp}}\} \tag{1}$$

, where W is a trainable model parameter, s_i is the linking score between query node and all other nodes inside the RSG. In practice, we focus on nodes that are imported into the file being predicted, as suggested in RepoBench [10]. The training loss of node link prediction for each query is computed as

$$\mathcal{L} = -\frac{1}{N_1} \sum_{i=1}^{N_1} y_i \log \hat{y}_i \text{ where } \hat{y}_i = \frac{1}{1 + e^{-s_i}} \tag{2}$$

In all experiments, we employ GraphSAGE [42] with L layers as the graph neural network (GNN) model to update the representation for each node based on the passage graph. The l-th layer of the GNN model updates the embedding of node i as follows:

$$Z_i^{(l)} = h\left(Z_i^{(l-1)}, \left\{Z_j^{(l-1)}\right\}_{(i,j)\in\mathcal{G}_1}\right) \tag{3}$$

where h is usually a non-linear learnable function which aggregates the embeddings of the node itself and its neighbor nodes. After re-ranking with linking scores, the final top-N_2 $(N_2 < N_1)$ contexts are sent for decoding. Suppose their indices are $\{g_1, g_2, \cdots, g_{N_2}\}$, the decoding process for final prediction $\mathbf{C}$ is:

$$\mathbf{C} = \text{ LLM } \left(Q, \left[\mathbf{P}_{g_1}; \mathbf{P}_{g_2}; \cdots; \mathbf{P}_{g_{N_2}}\right]\right) \tag{4}$$

where $\mathbf{P}_i$ is code representation of node $V_i \in V$

To train such a network to re-rank inside a repository, we solely optimize loss function defined in (1) on dataset $R = \left\{\left(\mathcal{G}^i, y^i_{\text{optimal}}, Q^i\right)\right\}$ with $\mathcal{G}^i$ is a RSG for a ith repository and y^i_{optimal} is the optimal context node corresponding to query Q^i.

Specifically, in the training set of RepoBench-R, for each sample, every snippet parsed from import statements is treated as a potential candidate for next-line prediction, for example, we have definitions of two following imported functions: "add" and "minus" as the candidate snippets and "add" as a gold snippet is the optimal context for prediction, here which is "add" is imported in "from src import add, minus" and is used in the nextline: "add(1,2)". These gold snippets are considered as optimal context node since it's the most relevant and decisive context to complete the code in the next line.

IV. Empirical Evaluation

A. Datasets and Procedure

We also follow [10] to use RepoBench as the main dataset for our evaluation pipeline due to its large scale and comprehensive, making it an ideal candidate for assessing repository-level code completion from multiple perspectives. RepoBench consists of three distinct subsets, each designed to evaluate different aspects of repo-level code completion. Each subset contains up to 12000 samples for evaluation.

1) **RepoBench-R** focuses on evaluating the retrieval of relevant code snippets, crucial for accurate code prediction. It assesses the model's ability to sift through extensive repository data to identify useful snippets for code completion, termed as the **Context Retrieval** task.
2) **RepoBench-C** is designed to predict the next line of code using provided in-file and cross-file contexts, testing a model's ability to predict precise code completion from available contexts.
3) **RepoBench-P** combines the challenges of RepoBench-R and C, testing a model's pipeline from snippet retrieval to code prediction, reflecting real-world auto-completion (denoted as the **End-to-End Code Completion** task).

Our work focuses on the creation of repository-level code graphs and retrieval strategies, primarily utilizes RepoBench-R and RepoBench-P. These subsets align with our goals to enhance context retrieval and code completion accuracy, directly showcasing our contributions.

Within these benchmarks, there are two settings [5] to thoroughly assess performance: **Cross-File-First** (XF-F) challenges a model to predict the first occurrence of a cross-file line, requiring adept handling of long-range contexts; **Cross-File-Random** (XF-R) masks a random cross-file line where prior usage might offer clues for prediction. These settings enable a robust evaluation on these code completion scenarios.

B. Empirical Methodology

1) Context Retrieval on RepoBench-R

We follow the methodology in [10] to define three baselines:

(a) **Random Retrieval**, where code snippets are chosen randomly, providing a basic comparison level. This process is repeated 100 times to average the results for consistency.
(b) **Lexical Retrieval**, which uses simple text comparison techniques like Jaccard Similarity and Edit Distance to find relevant snippets based on the code tokens.
(c) **Similarity-based Retrieval**, employing encoder models like CodeBERT, UnixCoder and OpenAI Text Embedding Models to generate code embeddings and Cosine Similarity to measure the semantic similarity between the cropped code and the candidate snippets. In REPOHYPER setting, we employ our pipeline, which leverages different encoder models (UniXCoder [12], CodeT5+-770M [11]) to encode

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

TABLE I
RESULTS OF REPOHYPER ON REPOBENCH-R DATASET. THE STUDIED ENCODER MODELS INCLUDE `CODEBERT-BASE` FOR CODEBERT, `UNIXCODER-BASE` FOR UNIXCODER, AND `CODET5P` FOR USING CODET5+ 770M PARAMETERS. REPOHYPER HAS UNIXCODER AND CODET5+ 770M AS THE BASE ENCODERS, RESPECTIVELY. NUMBERS ARE SHOWN IN PERCENTAGE (%), WITH THE BEST PERFORMANCE HIGHLIGHTED IN BOLD. `TEXT-EMBEDDING-LARGE-3` FOR OPENAI.

Retrieval	Model	Easy				Hard					
		XF-F		XF-R		XF-F			XF-R		
		acc@1	acc@3	acc@1	acc@3	acc@1	acc@3	acc@5	acc@1	acc@3	acc@5
Random		15.68	47.01	15.61	46.87	6.44	19.28	32.09	6.42	19.36	32.19
Lexical	Jaccard	20.82	53.27	24.28	54.72	10.01	25.88	39.88	11.38	26.02	40.28
	Edit	17.91	50.61	20.25	51.73	7.68	21.62	36.14	8.13	22.08	37.18
Similarity-based	CodeBERT	16.47	48.23	17.87	48.35	6.56	19.97	33.34	7.03	19.73	32.47
	UniXcoder	25.94	59.69	29.40	61.88	17.70	39.02	53.54	20.05	41.02	54.92
	CodeT5+	18.29	53.31	19.61	53.05	9.51	25.24	37.81	13.21	28.15	38.76
	OpenAI	28.14	64.24	31.15	65.29	18.23	42.01	60.39	23.78	45.67	58.53
REPOHYPER	UniXcoder	**32.56**	**68.91**	**33.79**	**69.67**	**25.81**	**49.51**	**61.19**	**27.12**	**51.12**	**63.23**
	CodeT5+	31.15	67.23	32.01	68.12	25.16	46.70	60.19	23.81	43.61	57.12

the query, construct semantic graph, node expansion, and link prediction to retrieve the contexts.

Evaluation Metrics. We also follow the methodology [10] use to Accuracy@k (acc@k) metric to evaluate performance on this task. For the easy subset of tasks, we assess performance using acc@1 and acc@3, while for the more challenging subset, we evaluate using acc@1, acc@3, and acc@5.

2) Code Completion on RepoBench-P

This end-to-end code completion task requires two components: Context Retrieval and Code Completion. Given our research's emphasis on developing novel context retrieval strategies, we examine how these strategies perform under various settings, using consistent code completion models for comparison. Specifically, we utilized GPT-3.5-Turbo and DeepSeek-Coder-33B [23] as our the LLM for code completion in our workflow.

We follow RepoBench [10] to define baselines according to different settings on the contexts: (1) **Gold-Only**, which uses only the 'gold snippet' for cross-file completions and leaves in-file completion contexts empty, testing a model's efficacy with minimal context. (2) **In-File-Only**, which uses maximum 30 lines up from prediction line in same file, indicates lower-bound performance without repo-level contexts. (3) **RepoBench**, which uses similarity-based search, UniXCoder, to retrieve top contexts, then prompts LLM for code completion.

We also assess snippet ranking strategies, **H2L** (High-to-Low) and **L2H** (Low-to-High), to see how the order of context relevance affects code completion performance. We also include **RepoCoder** [5], which is a method on the repo-level code completion task as another baseline. This method is run iteratively with Jaccard retrieval method and maximum iterations of 4.

Evaluation Metrics. We use Exact Match (EM) and CodeBLEU [43] to measure next-line completion accuracy as in [10].

C. Implementation Details

To construct the Repo-level Semantic Graph (RSG) for a repository, we first parse functions, methods, and classes using *tree-sitter*[1], a tool for generating abstract syntax trees (AST). We then extract code entities from the AST and integrate them into a semantic graph. Specifically, we use Tree-sitter [2] to parse functions, methods, classes out of Python files. After parsing these entities, we removed these entities from each file so the remaining codes in the file is not function or class. This ensures import statements and other statements (like main file) will be remained. In order to create *Import Relations* between script nodes and the imported nodes (functions, classes), we use built-in Abstract Syntax Tree (AST) module of Python to parse import statements, then identify which module in parsed entities is imported into the script node. For *Invoke Relations*, we use PyCG [3] to generate the call graph of the repository. Since, PyCG only works for Python3 repositories, and RepoBench contains a lot of Python2 repositories, we use Automated Python 2 to 3 code translation tool 2to3 [4] to translate these repositories into Python3, then apply PyCG to produce call graph. *Ownership* and *Encapsulate* relationships are straightforward to generate since the Tree-sitter allows us to identify which method belongs to which class exactly and we also parse functions, classes from each file then we also know which function, class belongs to which file exactly. For *Class Hierarchy Relations*, since Python is a language that

[1]https://github.com/tree-sitter/tree-sitter
[2]https://tree-sitter.github.io/tree-sitter/
[3]https://github.com/vitsalis/PyCG
[4]https://docs.python.org/3/library/2to3.html

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

needs to specify parent class in the implementation of the inherited class, we can use Tree-sitter to parse this parent class in declaration fields of the inherited class, then we can produce a class hierarchy edge between parent and child class.

In our methodology, we adopt *Pattern Search* for expansion with parameters set to a maximum depth $D = 4$, $M = 1000$, and $K = 3$. The number of prompt contexts for LLM completion, N_2, is dynamically selected to maximize token count within LLMs' context length limits. The Link Predictor is trained on training subset of RepoBench-R's gold context labels, with a text matching algorithm based on Jaccard distance used to align labels with RSG nodes. We use a homogeneous GraphSAGE network f with $L = 3$ GNN layers, optimized with Adam at a learning rate of 0.01 for 10 epochs, noting homogeneous networks performed adequately for RSG. Link Predictor is trained on 2 A100 GPUs in 6 hours.

V. Empirical Results

A. Performance on Context Retrieval

The results presented in Table I demonstrate the enhanced performance of RepoHyper when compared to similarity-based approaches . By implementing graph-based semantic search strategies, RepoHyper significantly outperforms the baseline methods, including those utilizing CodeBERT and UniXCoder, as well as our own tests with CodeT5+. Specifically, it achieves high improvements, with UniXcoder and CodeT5+ showing relative increases of nearly 26% and 72%, respectively, across various subsets and task scenarios.

B. Performance on Code Completion

TABLE II
Comparison of various context retrieval strategies (CR Strategy) on the end-to-end code completion task on RepoBench-P for Python using GPT-3.5-turbo-16k and DeepSeek-Coder-33B.

	CR Strategy	XF-F		XF-R	
		EM	CodeBLEU	EM	CodeBLEU
Gpt-3.5-turbo	In-File-Only*	26.35	33.14	36.31	44.01
	Gold-Only*	30.59	38.37	40.65	49.12
	RepoBench-L2H	37.51	46.19	49.3	57.20
	RepoBench-H2L	39.89	48.01	51.21	59.44
	RepoCoder	48.73	57.62	59.55	67.42
	RepoHyper-L2H	**52.76**	**61.49**	**64.06**	**71.59**
	RepoHyper-H2L	48.8	57.21	59.81	66.85
DeepSeek-Coder	In-File-Only*	25.46	32.39	34.23	42.13
	Gold-Only*	27.29	35.15	37.10	46.08
	RepoBench-H2L	34.02	43.08	45.96	53.62
	RepoBench-L2H	36.55	44.53	47.71	55.81
	RepoCoder	45.47	54.32	56.24	64.23
	RepoHyper-L2H	**49.49**	**58.48**	**59.03**	**66.98**
	RepoHyper-H2L	45.17	53.71	56.56	63.41

TABLE III
Sensitivity Analysis. D : maximum depth; M : maximum number of nodes; K : in kNN algorithm. During expansion, the search algorithm will expand K nodes to Coverage % of the number nodes of the repolevel semantic graph, then extract explored node into a sub-graph. There's a Hits % probability that there's an optimal context in this sub-graph. In kNN baseline, we use K=35% of size of the original graph.

Search Algorithm	Parameter Combination	Hit/Coverage	
		Hits	Coverage
Exhausted Search	D=4, M=1000, K=3	80%	40%
	D=4, M=200, K=4	78%	44%
	D=4, M=10000, K=1	72%	40.4%
	D=2, M=10000, K=1	34%	10%
	D=2, M=10000, K=2	47%	16%
	D=2, M=10000, K=8	73%	36%
Pattern Search	D=4, M=1000, K=3	73%	28%
	D=4, M=200, K=4	70%	29%
	D=4, M=10000, K=1	65%	31%
	D=2, M=10000, K=1	62%	14%
	D=2, M=10000, K=2	51%	8%
	D=2, M=10000, K=8	68%	21%
kNN	D=M=1, K=0.35*$\lvert\mathcal{G}\rvert$	53%	35%

Table II shows that RepoHyper emerges as the most effective strategy, consistently achieving the highest scores across both EM (Exact Match) and CodeBLEU metrics in both XF-F and XF-R settings, with notable scores such as 52.76% EM and 61.49% CodeBLEU with gpt-3.5-turbo, and 49.49% EM and 58.48% CodeBLEU with DeepSeek-Coder. Compared to the baseline strategies like Gold-Only, RepoBench-L2H/H2L, and RepoCoder, RepoHyper's strategies (both L2H and H2L) demonstrate superior performance. The comparison between L2H (Low-to-High relevance) and H2L (High-to-Low relevance) within RepoHyper indicates that prioritizing snippets from low to high relevance (L2H) offers a significant advantage over the reverse, particularly highlighting the high performance of RepoHyper-L2H strategy.

VI. Ablation Study

A. Sensitivity on Hyper-parameters and Effects

We evaluated various graph-based semantic search strategies, including Exhausted, Pattern, and the baseline kNN search, each with different hyper-parameters. Table III reveals that while Exhausted Search achieves high hit rates, Pattern Search offers a more efficient solution, reaching up to 73% hit rates by exploring only 28% of the nodes, compared to 36.7% needed by Exhausted Search for similar outcomes. This efficiency highlights Pattern Search's ability to identify relevant sub-graphs more efficiently, enhancing precision with less computational resources. Moreover, both Exhausted and Pattern Search strategies significantly outperform the kNN baseline in hit rates, while maintaining smaller, more focused sub-graphs for quicker inference and reduced noise. These results emphasize the importance of choosing the right search strategy and hyperparameter tuning to balance search thoroughness

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

and prediction efficiency. Even if we raise M (the maximum number of nodes), it does not assure a higher hit rate, as the search process may end upon reaching its maximum depth limit before evaluating all M nodes and vice versa.

B. Analysis on Retrieved Nodes

REPOHYPER is designed to retrieve both semantic similar code contexts, akin to similarity-based semantic search baselines, as well as relevant graph-based semantic code contexts like *Sibling* or *Import of Parent Class*. In this experiment, our objective is to validate the hypothesis that REPOHYPER performs better at retrieving both types of code contexts. To achieve this, we analyzed the retrieval performance for different types of repository-level contexts. Utilizing the ground-truth (optimal) context provided by RepoBench-R for each instance in the dataset, we manually examined these optimal contexts within the *Hard* subset of RepoBench-R test set. We then categorized them into different types of contexts, following the classification scheme introduced in [41]. These context types include *Parent Class*, *Sibling*, *Similar Name*, *Import of Sibling*, *Import of Similar Name*, *Child Class*, *Import of Parent Class*, and *Import of Similar Name*. It's worth noting that we excluded some context types mentioned in [41] due to the negligible number of instances with those context types in the dataset. We conducted this experiment with 100 instances per context type under the XF-R setting and measured the performance using acc@5 metric.

Figure 3 illustrates that REPOHYPER performs better in retrieving context types relevant to the structural properties of the code, such as Class Hierarchy (Parent Class or Import Sibling), compared to the Similarity-based Semantic Search baseline. This observation validates our earlier hypothesis. Note that although REPOHYPER consistently outperforms the similarity-based baseline in retrieving several context types, it still falls short in retrieving the similar name context type. This can be explained via the fact that during the kNN search phase of REPOHYPER, it uses only a small number of anchors (k) for expansion. Using a larger value of k can potentially improve the performance in retrieving similar snippets but may adversely affect the performance for other context types.

We follow definitions of different contex types in [41] and removed more of them :

- Parent Class (ParCls): code snippet taken from the parent class of the class that is having the target prediction line inside.
- Child Class (ChiCls): code snippet taken from the child class of the class that is having the target prediction line inside.
- Sibling (Sib): any code snippet in the files that are in the same directory as the current file (the file contains current target prediction line)
- Similar Name (SimN): take code snippet from files, functions, classes (objects) that have a similar name as the completing function, class or file. Similar names are determined by splitting the file name based on underscore or camel- case formatting and then matching parts of the filename. If one or more parts matches, two objects are considered to have similar names.
- Import Sibling (ImpSib): take code from the import objects (classes, functions) in the sibling files.
- Import Similar Name (ImpSimN): take code snippet from the import files used in the similar name files.
- Import Parent Class (ImpParCls): any code snippet from the import objects used in the parent class files. This implies the case when the child class is likely to re-use imported objects of its parent class.

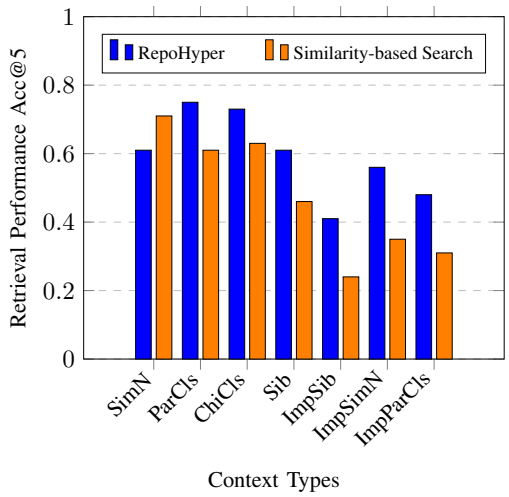

Fig. 3. Retrieval performance comparison between REPOHYPER and Similarity-based Semantic Search across different context types. We use kNN search within our RSG with UniXCoder encoder for encoding, this method is denoted as Similarity-based Semantic Search and REPOHYPER with same encoder.

C. Ablation Results

TABLE IV
ABLATION STUDY IN THE CODE RETRIEVAL TASK. WE USE THE REPOBENCH-R TEST SET WITH TWO SUBSETS, *Easy* AND *Hard*. ACC@3 IS USED AS THE MAIN METRIC. EXHAUSTED SEARCH IS DENOTED AS E.S, AND PATTERN SEARCH IS DENOTED AS P.S. FOR A COMBINATION WITH KNN AND EXPANSION STRATEGY, WE RE-RANK INSIDE THE EXTRACTED SUB-GRAPH USING COSINE SIMILARITY BETWEEN THE QUERY AND NODES IN THE SUB-GRAPH.

Models	Easy	Hard
(1) kNN	60.15	40.74
(2) w/ Exhausted Search	62.35	42.88
(3) w/ Pattern Search	64.23	44.50
(4) w/ E.S+Link Predictor	68.10	47.15
(5) w/ P.S+Link Predictor	69.12	47.83
(6) w/ P.S+Re-ranking	67.43	44.48

To evaluate the contributions of different components in our model, we conduct an ablation study on the code retrieval task. Specifically, we vary key elements such as the Search-then-Expand strategies and the Link Predictor. The results,

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

presented in Table IV, provide insights into the impact of these components under the *Easy* and *Hard* subsets of the REPOBENCH-R test set.

From Table IV, we make the following observations:

- **Search-then-Expand Strategies**: Pattern Search outperforms Exhausted Search in both the *Easy* and *Hard* subsets, achieving accuracies of 64.23% and 44.50%, respectively. This indicates that Pattern Search is more efficient at identifying relevant contexts without including excessive noise, which is a common drawback of Exhausted Search.
- **Link Predictor Contribution**: The inclusion of the Link Predictor significantly improves accuracy for both search strategies. For example, Pattern Search combined with the Link Predictor achieves the highest accuracy across both subsets (69.12% on *Easy* and 47.83% on *Hard*). This highlights the Link Predictor's critical role in refining and ranking the retrieved contexts.
- **Effect of Re-ranking**: While re-ranking based on cosine similarity in the extracted sub-graph (P.S+Re-ranking) offers improvements over standalone search strategies, its performance falls short of the combined P.S+Link Predictor setup. This suggests that a more sophisticated ranking mechanism, such as the Link Predictor, is essential for achieving state-of-the-art results.

In summary, the results underscore the importance of combining efficient search strategies with robust ranking mechanisms. Pattern Search, when paired with the Link Predictor, emerges as the optimal configuration, demonstrating the ability to balance retrieval efficiency and context relevance.

D. Qualitative Examples

In Fig. 4, we present a sample where the similarity-based retrieval method fails, illustrating its limitations. Using UniXCoder as the main encoder, this example is drawn from the REPOBENCH-R Cross-File First subset. The orange block in the figure highlights the ground-truth snippet for `fields_desc` in the `SMB2_Error_Response` class. Similarity-based methods mistakenly prioritize the `GSSAPI_BLOB` class due to its structural resemblance to the current in-file context. This results in incorrect context identification and misinterpretation.

In contrast, REPOHYPER utilizes a graph-based approach to effectively identify and link the most relevant code elements, such as `SMB2_Read_Request` and `ByteField`, by leveraging broader semantic and structural relationships within the repository. This enables accurate context expansion and ensures proper function completion, demonstrating the robustness of the graph-based retrieval method over the state-of-the-art traditional similarity-based techniques.

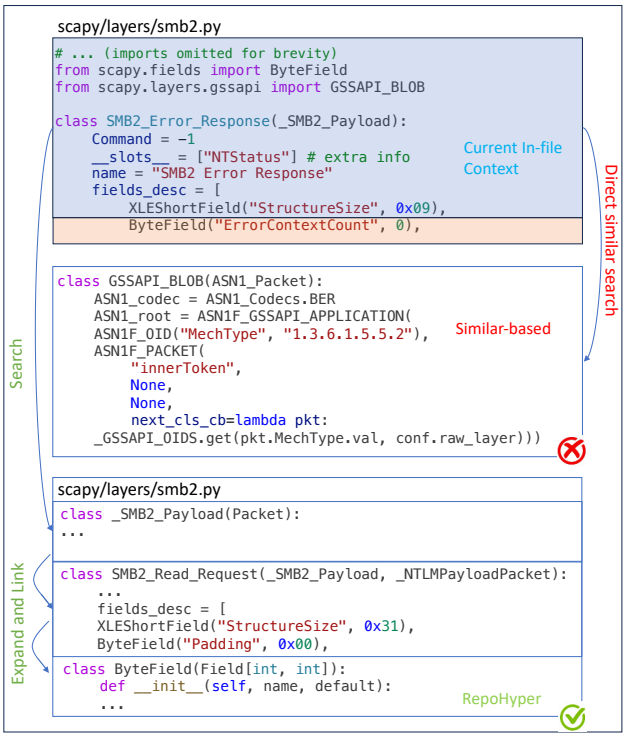

Fig. 4. Sample ID 1430 in repository secdev/scapy

VII. CONCLUSIONS, LIMITATIONS & FUTURE WORK

This paper presents REPOHYPER, a framework aimed at addressing the challenges of repository-level code completion. By leveraging the Repo-level Semantic Graph (RSG), the Search-then-Expand strategy, and a Link Predictor, REPOHYPER demonstrates improvements in the precision and relevance of retrieved contexts for code completion tasks. Our evaluations on the REPOBENCH benchmark indicate that these components contribute to measurable gains in Exact Match (EM) and CodeBLEU metrics, highlighting the potential of graph-based semantic representations combined with programmatically guided retrieval techniques.

While the results are promising, this work has certain limitations that warrant further exploration. The Pattern Expansion strategy relies on manually selecting the most frequent path types from kNN searched nodes to target nodes, which may not be optimal and could vary across different programming languages. Automating this process through a learnable algorithm for navigating the RSG, as suggested by [44], could enhance its adaptability and effectiveness.

In summary, REPOHYPER showcases an approach that combines semantic graph representations and retrieval strategies to improve repository-level code completion. Our evaluations show that REPOHYPER markedly outperforms existing techniques in repository-level code completion, showcasing enhanced accuracy when compared to baselines. While further work is needed to expand its scope and robustness, we hope this study offers valuable insights and a stepping stone for future research in AI-assisted software engineering tools.

ACKNOWLEDGMENTS

Tien N. Nguyen was supported in part by the US National Science Foundation (NSF) grant CNS-2120386 and the National Security Agency (NSA) grant NCAE-C-002-2021.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restrictio

REFERENCES

[1] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," vol. 35, pp. 16 344–16 359, 2022.

[2] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[3] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," *arXiv preprint arXiv:2108.12409*, 2021.

[4] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending context window of large language models via positional interpolation," *arXiv preprint arXiv:2306.15595*, 2023.

[5] Y. Liu, Y. Wang, X. Zhang, J. Li, and X. Liu, "Repocoder: Repository-level code completion through cross-file context retrieval," *arXiv preprint arXiv:2303.12570*, 2023.

[6] D. Liao, S. Pan, Q. Huang, X. Ren, Z. Xing, H. Jin, and Q. Li, "Context-aware code generation framework for code repositories: Local, global, and third-party library awareness," *arXiv preprint arXiv:2312.05772*, 2023.

[7] D. Shrivastava, D. Kocetkov, H. de Vries, D. Bahdanau, and T. Scholak, "Repofusion: Training code models to understand your repository," 2023.

[8] L. A. Agrawal, A. Kanade, N. Goyal, S. K. Lahiri, and S. K. Rajamani, "Guiding language models of code with global context using monitors," *arXiv preprint arXiv:2306.10763*, 2023.

[9] D. Shrivastava, H. Larochelle, and D. Tarlow, "Repository-level prompt generation for large language models of code," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 693–31 715.

[10] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems," 2023.

[11] Y. Wang, H. Le, A. Gotmare, N. Bui, J. Li, and S. Hoi, "CodeT5+: Open code large language models for code understanding and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 1069–1088. [Online]. Available: https://aclanthology.org/2023.emnlp-main.68

[12] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," *arXiv preprint arXiv:2203.03850*, 2022.

[13] N. D. Bui, H. Le, Y. Wang, J. Li, A. D. Gotmare, and S. C. Hoi, "Codetf: One-stop transformer library for state-of-the-art code LLM," *arXiv preprint arXiv:2306.00029*, 2023.

[14] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

[15] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021.

[16] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[17] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song *et al.*, "Measuring coding challenge competence with apps," *arXiv preprint arXiv:2105.09938*, 2021.

[18] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," 2023.

[19] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "Codegen2: Lessons for training llms on programming and natural languages," *arXiv preprint arXiv:2305.02309*, 2023.

[20] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, Z. Wang, L. Shen, A. Wang, Y. Li, T. Su, Z. Yang, and J. Tang, "CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Evaluations on HumanEval-X," 2023.

[21] N. D. Bui and L. Jiang, "Hierarchical learning of cross-language mappings through distributed vector representations for code," in *Proceedings of the 40th International Conference on Software Engineering: New Ideas and Emerging Results*, 2018, pp. 33–36.

[22] V. Jayasundara, N. D. Q. Bui, L. Jiang, and D. Lo, "TreeCaps: Tree-structured capsule networks for program source code processing," *arXiv preprint arXiv:1910.12306*, 2019.

[23] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, "DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence," 2024.

[24] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, M. Davaadorj, J. Lamy-Poirier, J. Monteiro, O. Shliazhko, N. Gontier, N. Meade, A. Zebaze, M.-H. Yee, L. K. Umapathi, J. Zhu, B. Lipkin, M. Oblokulov, Z. Wang, R. Murthy, J. Stillerman, S. S. Patel, D. Abulkhanov, M. Zocca, M. Dey, Z. Zhang, N. Fahmy, U. Bhattacharyya, W. Yu, S. Singh, S. Luccioni, P. Villegas, M. Kunakov, F. Zhdanov, M. Romero, T. Lee, N. Timor, J. Ding, C. Schlesinger, H. Schoelkopf, J. Ebert, T. Dao, M. Mishra, A. Gu, J. Robinson, C. J. Anderson, B. Dolan-Gavitt, D. Contractor, S. Reddy, D. Fried, D. Bahdanau, Y. Jernite, C. M. Ferrandis, S. Hughes, T. Wolf, A. Guha, L. von Werra, and H. de Vries, "StarCoder: may the source be with you!" 2023.

[25] A. T. Dau, H. T. Dao, A. T. Nguyen, H. T. Tran, P. X. Nguyen, and N. D. Bui, "XMainframe: A large language model for mainframe modernization," *arXiv preprint arXiv:2408.04660*, 2024.

[26] N. Muennighoff, Q. Liu, A. Zebaze, Q. Zheng, B. Hui, T. Y. Zhuo, S. Singh, X. Tang, L. Von Werra, and S. Longpre, "Octopack: Instruction tuning code large language models," *arXiv preprint arXiv:2308.07124*, 2023.

[27] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.

[28] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," *arXiv preprint arXiv:2306.08568*, 2023.

[29] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," 2024.

[30] Y. Wang, X. Zhang, J. Li, Y. Liu, and X. Liu, "RLPG: A Reinforcement Learning Based Code Completion System with Graph-Based Context Representation," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 1119–1129. [Online]. Available: https://doi.org/10.1145/3468264.3468573

[31] Y. Ding, Z. Wang, W. U. Ahmad, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, "CoCoMIC: Code Completion By Jointly Modeling In-file and Cross-file Context," *arXiv preprint arXiv:2212.10007*, 2022.

[32] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth *et al.*, "Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion," *arXiv preprint arXiv:2310.11248*, 2023.

[33] K. Zhang, J. Li, G. Li, X. Shi, and Z. Jin, "Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges," *arXiv preprint arXiv:2401.07339*, 2024.

[34] H. N. Phan, T. N. Nguyen, P. X. Nguyen, and N. D. Q. Bui, "Hyperagent: Generalist software engineering agents to solve coding tasks at scale," 2024. [Online]. Available: https://arxiv.org/abs/2409.16299

[35] W. Norcliffe-Brown, E. Vafeias, and S. Parisot, "Learning conditioned graph structures for interpretable visual question answering," *ArXiv*, vol. abs/1806.07243, 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:49317766

[36] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *ArXiv*, vol. abs/1810.00826, 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:52895589

[37] Z. Chen, L. Chen, S. Villar, and J. Bruna, "Can graph neural networks count substructures?" 2020.

[38] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=BJOFETxR-

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restriction

[39] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, *Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[40] I. Abdelaziz, J. Dolby, J. McCusker, and K. Srinivas, "A toolkit for generating code knowledge graphs," 2021.

[41] D. Shrivastava, H. Larochelle, and D. Tarlow, "Repository-level prompt generation for large language models of code," in *ICML 2022 Workshop on Knowledge Retrieval and Language Models*, 2022. [Online]. Available: https://openreview.net/forum?id=bUDmRzeh3PT

[42] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *NIPS*, 2017.

[43] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "CodeBLEU: a method for automatic evaluation of code synthesis," *arXiv preprint arXiv:2009.10297*, 2020.

[44] S. Moon, P. Shah, A. Kumar, and R. Subba, "OpenDialKG: Explainable conversational reasoning with attention-based walks over knowledge graphs," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Màrquez, Eds. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 845–854. [Online]. Available: https://aclanthology.org/P19-1081

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 18:23:52 UTC from IEEE Xplore. Restriction